Assignment 2

Aditya Sarkar

Tulane Dr, Hyattsville, MD | October 19, 2026

Problem Overview

Implement the function `arithmetic_intensity(graph: Graph) -> dict[str, Any]` to compute the **arithmetic intensity** (FLOPs per byte moved) for each individual layer and across the entire model graph.

Arithmetic intensity characterizes the operational workload per unit of memory transferred:

$$\text{Arithmetic Intensity} = \frac{\text{FLOPs}}{\text{Bytes Moved}}$$

Every layer incurs two fundamental costs: arithmetic operations and memory traffic. Their ratio governs whether a given layer is compute-bound or memory-bandwidth-bound on target hardware.

Memory Traffic Model (DRAM Bounds)

Compute memory traffic under a standard **pessimistic DRAM model**:

- **No Cache Reuse:** Every tensor accessed by a layer must be read from or written to main memory once:

$$\text{Bytes Moved} = \text{Input Bytes} + \text{Weight/Buffer Bytes} + \text{Output Bytes}$$

- **No Operator Fusion:** Adjacent layers do not fuse intermediate activations in registers or shared cache.

Function Specification

```
def arithmetic_intensity(graph: Graph) -> dict[str, Any]:
    ...
```

Inputs

- `graph`: A `Graph` object loaded from the input model description file.
  - `graph.name`: `str` — Name of the model.
  - `graph.precision`: `str` — Default precision of external inputs (e.g., `"fp32"`).
  - Iterating over `graph` yields `Layer` instances in topological execution order.

Each `ly: Layer` in `graph` exposes:

- `ly.name`: `str` — Unique layer identifier.
- `ly.kind`: `str` — Operator classification (e.g., `"conv"`, `"linear"`, `"bn"`, `"relu"`, `"add"`).
- `ly.reads`: `list[str]` — Identifiers of producer layers feeding this layer.
- `ly.act_dtype`: `str` — Precision of activations produced by this layer.
- `ly.weight_dtype`: `str` — Precision of parameter weights.
- `ly.in_elements`: `int` — Total scalar elements in one input activation tensor.
- `ly.out_elements`: `int` — Total scalar elements in the output tensor.
- `ly.out_shape`: `tuple[int, ...]` — Output shape tuple (where `out_shape[0]` denotes output channel count).

Available global constants and utility functions:

- `dtype_bytes(dtype: str) -> int`: Byte width per element (e.g., 4 for `"fp32"`, 2 for `"fp16"`).
- `_layer_parameters(ly: Layer) -> int`: Total parameter count for `ly`.
- `_layer_macs(ly: Layer) -> int`: Total Multiply-Accumulate operations for `ly`.
- `FLOPS_PER_MAC = 2`: Scaling factor (1 MAC = 2 FLOPs).
- `BN_BUFFERS_PER_CHANNEL = 2`: Two non-parameter state buffers per channel (`running_mean`, `running_var`).
- `BUFFER_DTYPE = "fp32"`: Buffer data type.
- `computed(value, message, **metadata)`: Helper wrapping findings into a standardized dictionary.

Output Format

The return value must be generated by calling `computed(...)`:

```
{
    "type": "computed",
    "value": float,              # Whole-model intensity rounded to 4
       decimals
    "message": str,              # f"{graph.name}: FLOPs over bytes,
       every byte counted once"
    "per_layer": {
        "<layer_name>": {
            "flops": int,        # FLOP count for this layer
            "bytes": float,      # Input + weights/buffers + output
               bytes
            "intensity": float   # flops / bytes rounded to 4 decimals
                (0.0 if bytes == 0)
        },
        ...
    },
    "total_flops": int,          # Aggregate FLOPs across all layers
    "total_bytes": float,        # Aggregate bytes transferred across
       all layers
    "flops_per_mac": 2,
    "cache_reuse_assumed": False,
    "note": "no␣fusion␣and␣no␣cache␣reuse;␣both␣would␣lower␣the␣byte␣
       column"
}
```

Key Implementation Requirements

1. **Input Tensor Dtype Resolution:** An input activation tensor's data type is defined by the *producing* layer's `act_dtype`, not the current layer's.
   - If `ly.reads` is non-empty, retrieve the producer named in `ly.reads[0]`.
   - If `ly.reads` is empty and $i > 0$, resolve to the immediate predecessor's `act_dtype`.
   - If $i = 0$ (graph input), fall back to `graph.precision`.
2. **Dual-Input Elementwise Operators (`ly.kind == "add"`):** If `ly.kind == "add"` and `len(ly.reads) == 2`, the layer reads two incoming activation streams; scale input bytes by 2:
$$\text{in_b} = 2 \times \text{ly.in_elements} \times \text{dtype_bytes(in_dtype)}$$
3. **BatchNorm State Buffers (`ly.kind == "bn"`):** Include memory traffic for running statistics in addition to learnable parameters (γ, β):

   buffer_bytes = BN_BUFFERS_PER_CHANNEL×ly.out_shape[0]×dtype_bytes(BUFFER_DT

4. **Zero-FLOP Operations (e.g., ReLU):** Layers without arithmetic operations perform 0 FLOPs but still read and write memory. Their arithmetic intensity must evaluate to 0.0, not omitted or `None`.